Evaluating Synthetic Data Training, Model Architecture, and Quantization for CCTV Analytics on Raspberry Pi 5

Tadhg Roche[1]

[1]School of Computer Science, Technological University Dublin, Email: C22348761@mytudublin.ie

March 2026

Abstract

Most CCTV systems rely on cloud processing or expensive hardware to perform intelligent video analysis, limiting their use in cost-sensitive or network-constrained environments. This study evaluates how training data sources, model architecture, INT8 quantization, and synthetic-to-real data mixing affect person detection accuracy and inference speed on the Raspberry Pi 5. Models trained on real CCTV data significantly outperform those trained on synthetic data alone (mAP@0.50 of 0.931 vs 0.719). A comparison of four edge-optimised architectures shows that lightweight CNN-based YOLO models outperform the transformer-based RF-DETR-N on this task, with YOLO26n achieving the highest localisation accuracy (mAP@0.50:0.95 of 0.745). After INT8 quantization, YOLO26n achieves the fastest inference on the Raspberry Pi 5 at 5.32 FPS with a model size of just 4.6 MB, while retaining an mAP@0.50:0.95 of 0.712. Across multiple training runs with different random subsets of images, training on real data alone produces the highest accuracy, with only very small proportions of synthetic data performing comparably and no mixture improving over the real-only baseline. These results show that YOLO26n quantized to INT8, trained on real CCTV data, offers the best balance of accuracy, speed, and size for low-cost, privacy-friendly CCTV person detection on edge hardware.

1 Introduction

Many CCTV systems still use simple motion detection, which reacts to movement in the camera's view [1]. However, it cannot tell what is moving, so harmless changes such as shadows, lights switching on, or weather often trigger false alarms [2]. These systems are cheap and easy to set up, but they lack real understanding of what they see, leading to unreliable security events [2].

Newer AI-based systems can recognise people or vehicles instead of just movement [3]. However, most systems either rely on cloud servers or expensive cameras with embedded AI hardware.

Cloud-based systems need a strong, stable internet connection to send footage for analysis, which can introduce delays or interruptions in areas with poor connectivity [4]. Such delays can be critical in security situations, where every second matters [5]. Cameras with built-in AI respond faster because they process video locally, analysing each frame on the device itself to identify objects such as people, instead of sending footage to cloud servers for analysis. However, they are costly and often use simple heuristic algorithms rather than deep-learning models, which limits accuracy in real-world conditions [6]. As a result, achieving affordable and reliable real-time monitoring remains difficult.

Edge computing refers to processing data close to where it is collected, for example on a small local computer (edge device) placed next to the camera, rather than sending all information to the cloud [7, 8]. This approach provides a practical solution to the limitations of cloud or on-device AI by enabling real-time analysis without high costs or network dependence. Devices such as the Raspberry Pi or Jetson Nano can run AI tasks independently, reducing network load, avoiding internet delays, and improving privacy since the footage never leaves the premises [8]. This enables real-time AI analysis, where events are processed as they occur, on affordable hardware, making it suitable for small-scale CCTV setups.

Edge AI is often used in modern CCTV systems as part of a two-stage approach: a smaller model on the device quickly flags possible events, while a more powerful model running on a local server can verify detections and filter out false alarms [8–10].

Object detection refers to the process of identifying and locating specific objects, such as people or vehicles, within video footage [11]. This technique plays a key role in AI-based edge CCTV systems, where cameras analyse scenes in real time rather than relying solely on motion detection. Most modern systems rely on deep learning [12], a branch of artificial intelligence that learns patterns from large image datasets to automatically recognise objects. In this process a model, meaning a learning system that has been trained on many example images, is used to make predictions on new, unseen images. Widely used deep learning models such as YOLO (You Only Look Once) are often applied for this purpose, as they can detect and recognise multiple objects in real time [13]. However, not all detection models are equally suited to resource-constrained devices. Different architectures vary in their parameter counts, computational costs, and how well they balance accuracy with inference speed. Selecting the right architecture is therefore an important consideration when deploying detection models on edge hardware.

Balancing accuracy and speed is a key challenge when running AI models on small devices with limited processing power [14]. Several research studies have compared deep learning models on edge devices to understand these trade-offs. Lightweight models such as YOLOv8-Nano and EfficientDet-Lite achieved the best balance between speed and accuracy when benchmarked on edge devices including the Raspberry Pi and Jetson Orin Nano, delivering real-time performance without major accuracy loss [15]. Other work showed that quantization [16], a model-compression method that reduces model size and speeds up inference so the same model can run faster on small devices, can significantly improve performance while maintaining similar

accuracy [17].

Another major challenge is training data. Real CCTV footage is difficult to collect due to privacy and security restrictions. In such cases, synthetic data can be generated using AI-based image generation techniques that recreate realistic scenes for model training [18, 19]. Studies have shown that synthetic data can achieve results close to real-world datasets [20–22], and that synthetic pedestrian datasets perform particularly well for person detection [23]. However, it is still unclear whether this holds true for CCTV-style footage, where lighting, camera angles, and quality vary widely.

Given these challenges and gaps in existing research, this study investigates three main research questions:

Research Question 1: *Can synthetic data match or supplement real CCTV data for person detection?*

This question examines whether models trained on synthetic CCTV images can achieve performance comparable to models trained on real footage, and whether mixing synthetic and real data can improve performance over real data alone. It is motivated by the difficulty of collecting real CCTV data due to privacy and legal restrictions, and by the increasing availability of generative models capable of producing realistic-looking scenes.

Research Question 2: *Which edge-optimised detection architecture provides the best accuracy for person detection on real CCTV footage?*

This question compares lightweight CNN-based and transformer-based detection architectures on the same real CCTV dataset to identify which is best suited to single-class person detection on a small dataset. While published benchmarks typically report results on large, multi-class datasets, it is unclear which approach performs best on a small, focused CCTV dataset where the deployment target is a low-power device.

Research Question 3: *What is the impact of INT8 quantization on detection accuracy and inference speed on the Raspberry Pi 5?*

This question evaluates how exporting trained models to ONNX format and applying INT8 quantization affects accuracy, model size, and inference speed when deployed on a Raspberry Pi 5. Edge devices have limited memory and processing power, so understanding the trade-off between speed gains and accuracy loss is essential for selecting a suitable model for real-time deployment.

The remainder of this report is organised as follows. Section 2 introduces the background concepts needed to understand the study, including object detection, the architectures evaluated, synthetic data, edge computing, and quantization. Section 3 describes the dataset construction process and the shared training configuration used across all experiments. Section 4 presents the four experiments that address the research questions, including their methods, results, and analysis. Section 5 interprets the findings in relation to each research question and discusses their practical implications and limitations. Section 6 summarises the main outcomes of the

study, and Section 7 outlines directions for future work.

2 Background

This section provides the background needed to understand the study. It introduces how computers detect objects in images, the main types of detection models, the application of CCTV analytics, the use of synthetic data, and the challenges of running these models on small, low-power devices.

2.1 Computer Vision

Computer vision is a field of artificial intelligence that enables computers to interpret and analyse visual information from images and videos [24]. It underpins many systems used in areas such as surveillance, robotics, healthcare, and autonomous technologies. The field encompasses several core tasks that address different forms of visual understanding, with object detection being among the most widely used.

2.1.1 Object Detection

Object detection identifies the objects present in an image and determines their locations [11]. Each detection consists of a bounding box, represented by four coordinates that show where the object appears, and a category label describing what the object is [13]. The task combines localisation, which finds the region containing the object, and classification, which assigns the correct category to that region [11]. These capabilities support applications in robotics, autonomous systems, and visual surveillance where timely and reliable scene understanding is essential [4, 9].

Modern object detection systems are trained using deep-learning models that learn from annotated images. Ground truth refers to the correct class labels and bounding-box coordinates provided during annotation. During training, the model receives an input image and predicts both the label and the bounding-box coordinates. These predictions are compared with the ground truth to calculate the loss, which measures how far the predictions differ from the correct values. The model updates its internal parameters to reduce this loss, gradually improving its ability to recognise objects and locate them accurately. Once training is complete, the process of using the trained model to make predictions on new, unseen images is called inference.

Most object detection models begin with a pretrained model that has already learned general visual features from large datasets such as COCO [25]. COCO is a large-scale dataset containing images of everyday scenes annotated with object categories and bounding boxes. Using a pretrained model provides a strong starting point because many useful visual patterns have already been learned, reducing the amount of training data and time required for the task.

Evaluation Metrics Object detection systems must determine both what objects are present in an image and where they appear. Evaluation therefore relies on metrics that measure these two aspects: how accurately the system identifies objects and how precisely it localises them.

The following metrics are the standard measures used to assess these components of detection performance.

Intersection over Union (IoU). IoU assesses localisation quality by comparing the model's predicted bounding box with the ground-truth box. It measures how much the two boxes overlap relative to the total area covered by both. A higher IoU indicates that the predicted box is closer to the correct location. A prediction is considered correct if the IoU exceeds a chosen threshold, with 0.5 being the most commonly used because it provides a reasonable balance between strict and lenient localisation requirements.

In this context, the *Area of Overlap* is the region where the predicted box and the ground-truth box intersect, while the *Area of Union* is the total area covered by both boxes combined.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Precision. Precision measures how many of the detections produced by the system are correct [26]. It is calculated as the proportion of *True Positives* (correct detections) out of all predicted positives. A *False Positive* occurs when the model reports an object that is not actually present in the image. High precision indicates that the detector produces few false alarms.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Recall. Recall measures how many of the real objects in the dataset are successfully detected [26]. It is calculated as the proportion of correctly detected objects among all objects that should have been detected. In this context, a *True Positive* is an object that the model correctly detects, while a *False Negative* is a real object that the model fails to detect. High recall indicates that the detector misses fewer objects.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}.$$

Confusion Matrix. A confusion matrix summarises the outcomes of a classifier by comparing its predictions to the ground-truth labels [27]. Each row represents what the model predicted, and each column represents the actual class. The diagonal cells contain correctly classified instances (true positives and true negatives), while the off-diagonal cells show the errors (false positives and false negatives). In object detection, the confusion matrix is useful for understanding whether a model tends to miss objects (high false negatives) or invent detections that are not actually present (high false positives). These two types of error correspond directly to lower recall and lower precision, respectively.

Mean Average Precision (mAP). Average Precision (AP) is a measure of detection performance that reflects how well a model balances precision and recall. In object detection, AP is calculated at several Intersection over Union (IoU) thresholds to evaluate localisation performance under different levels of strictness. Lower IoU thresholds, such as 0.50, are more lenient

and count a detection as correct even when the predicted bounding box is not perfectly aligned. Higher thresholds, such as 0.95, require very accurate localisation. Using multiple thresholds provides a more complete assessment of detection quality by capturing both coarse and highly precise detections.

Mean Average Precision (mAP) is the average of all AP values computed across these IoU thresholds:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} \text{AP}_i,$$

where N is the total number of IoU thresholds used, and AP_i is the Average Precision calculated at the ith threshold.

2.1.2 Technologies for Visual Recognition

Computer vision systems rely on models that learn to recognise patterns in images so they can make reliable predictions on new visual data [12]. The internal structure and design of a model is referred to as its architecture, and different architectures process images in different ways, which affects both accuracy and speed. Training these models requires large collections of images paired with correct labels that describe the content of each image. In object detection, labels specify both the category and the location of each object using bounding boxes. These labelled examples guide the learning process and allow the model to understand how different objects and scenes typically appear.

During training, the model compares its own predictions with the correct labels and adjusts its internal parameters so that its future predictions become more accurate. These internal parameters act as adjustable settings that determine how the model interprets patterns in the images. When a prediction is incorrect, the model updates these settings so that it becomes less likely to repeat the same mistake. Repeating this process across many examples allows the model to gradually improve and make more reliable predictions on unseen images.

Common Training Parameters. Training a computer vision model involves showing it many labelled images so that it can gradually learn the patterns needed for the task. Several settings, known as hyperparameters, control how this training process is carried out, and the values chosen for them have a significant influence on how effectively the model learns.

The training data is divided into small groups called *batches*, which makes training manageable and helps the model learn in steady steps. A complete pass through all training images is known as an *epoch*, and models usually require several epochs to improve consistently. The *image size* determines the resolution of the pictures used during training, with larger images providing more detail but requiring more computational power.

Two other important settings control how the model updates itself during training. The *learning rate* determines how much the model adjusts its internal values each time it learns from a batch.

If the value is too high, the model can behave unpredictably, while if it is too low, the learning process becomes slow. The *optimiser* is the algorithm that decides how the model's parameters are updated. Different optimisers use different strategies for adjusting the learning rate and update direction; for example, AdamW is a widely used optimiser that adapts the learning rate for each parameter individually, which often leads to faster and more stable training.

Finally, two settings affect how the data and randomness are managed. A *random seed* is a number used to initialise the random processes involved in training, such as the order in which images are shown to the model and the initial parameter values. Setting a specific seed makes the training process reproducible, and training with multiple different seeds allows the variability between runs to be measured. Datasets are also often prepared using *stratified sampling*, where images are selected from each subgroup in proportion to its size so that the overall composition of the dataset is preserved in each split.

Pretrained Models Pretrained models are computer vision systems that have already been trained on large, diverse image datasets [28]. Because they have learned general visual patterns from extensive data, these models provide a strong starting point for many tasks and avoid the need to train a new model from the beginning. They are widely used because collecting and labelling such large datasets is costly and time-consuming, while pretrained models make this existing knowledge readily available. These models can then be adapted to a new domain through fine-tuning, where their parameters are gradually adjusted using a smaller, task-specific dataset. Fine-tuning allows the model to retain what it has learned while becoming familiar with the characteristics of the new data.

CNN-Based Detection Models. Most modern object detection models are built using convolutional neural networks (CNNs). A CNN processes an image by sliding small filters across it, where each filter detects a specific local pattern such as an edge, a texture, or a shape. These filters operate on small regions of the image at a time, meaning the model builds its understanding by combining many local patterns into a larger picture. Because each filter only looks at a small area, CNNs are computationally efficient and well suited to tasks where local visual features such as the outline of a person or the shape of a face are the most important cues for detection [12].

YOLO Models. YOLO (You Only Look Once) is a family of CNN-based object detection models designed to make predictions quickly and efficiently [13]. These models were developed to overcome the slower processing speeds of earlier approaches by focusing on fast, streamlined detection. YOLO processes the entire image in a single pass rather than examining regions one at a time, which makes it particularly fast. The YOLO family includes many versions with different sizes. The smallest variants, known as "nano" models, have very few parameters and are designed specifically for deployment on devices with limited processing power. YOLO is widely used in situations where detections must be made immediately, such as video monitoring and other real-time applications, because it offers a strong balance between accuracy and processing speed.

Transformer-Based Detection Models. An alternative approach to object detection uses transformer models, which were originally developed for language processing and later adapted for vision tasks [29]. Unlike CNNs, which process small local regions of an image, transformers use a mechanism called attention that allows every part of the image to consider information from every other part. This gives transformers the ability to capture relationships across the entire image, which can be useful when objects are partially hidden or when understanding the broader scene helps identify what is present.

However, this global processing comes at a cost. Because every part of the image attends to every other part, the computational work grows quickly as image size increases. Transformer models also tend to have significantly more parameters than CNN models of similar capability, which means they require more memory and more training data to learn effectively. On large, diverse datasets with many object categories, transformers can achieve strong results. On smaller, more focused datasets, their large parameter counts can lead to overfitting, where the model memorises the training data rather than learning general patterns [29].

RF-DETR is one such transformer-based detector designed by Roboflow for real-time use. Its published benchmarks report fast inference on high-end NVIDIA graphical processing units (GPUs), which are specialised hardware designed to perform the large number of mathematical operations required by deep learning models much faster than standard CPUs. However, transformer models generally run much slower on CPUs because the attention mechanism lacks the hardware-level optimisation that CNNs benefit from on most processors.

Models for Real-Time Performance. Real-time performance is important in situations where visual information needs to be interpreted immediately, such as security monitoring or other time-critical applications. Models designed for real-time operation aim to process frames quickly enough to keep up with continuous video streams, providing predictions with minimal delay. They often use simplified architectures that allow them to operate at high speed while still maintaining reliable detection quality. For edge devices with limited processing power, CNN-based models such as the YOLO family are the most common choice because their local operations map efficiently to the hardware available on small processors.

2.1.3 Training Frameworks and Computing Environments

Modern computer vision models are trained using software frameworks that simplify and organise the learning process. These frameworks handle many routine tasks such as preparing images, sending them through the model, and keeping track of training progress. By automating these steps, they make it easier to train complex models reliably without requiring low-level implementation details.

Ultralytics [30] is one such framework commonly used in object detection. It provides a straightforward way to load pretrained models, apply custom datasets, and fine-tune a model on new data. The framework automatically manages important components of training, including data loading, parameter updates, and metric reporting. Because of this, Ultralytics is often used

when researchers want a practical and efficient method of adapting modern detection models to specialised tasks.

Frameworks such as Ultralytics use PyTorch as their underlying engine. PyTorch is a widely used open-source library that handles the mathematical operations involved in training and running deep learning models. When a model is trained, PyTorch stores the learned parameters in its own format. However, PyTorch is a large library that requires significant computing resources and is not designed to run efficiently on small edge devices. To deploy a trained model on hardware such as the Raspberry Pi, the model is typically exported to ONNX (Open Neural Network Exchange), a standardised format that allows models to be transferred between different software tools and hardware platforms. Once in ONNX format, the model can be run using ONNX Runtime, a lightweight inference engine that supports a wide range of devices including ARM-based processors, the type of low-power processor used in devices such as the Raspberry Pi and most modern smartphones. This export step separates the model from the training framework so that it can run independently on the target device.

Training deep learning models requires large numbers of numerical operations, which GPUs perform much more quickly than standard CPUs. Cloud services such as Vast.ai [31] provide access to high-performance GPU machines on demand, allowing researchers to rent powerful hardware for the duration of training without the cost of owning it. This ensures consistent hardware across all training runs and avoids the limitations of local machines.

2.2 CCTV Analytics

CCTV analytics refers to the use of automated methods to interpret and analyse video captured by surveillance cameras. These systems aim to extract useful information from continuous video streams, such as identifying people, detecting relevant events or monitoring activity within a scene. Before the introduction of modern computer vision techniques, most CCTV systems relied on classical motion detection, which detected movement by comparing pixel changes between consecutive frames [1, 32]. While effective for simple monitoring, these methods could not distinguish meaningful activity from irrelevant changes such as shadows, lighting variations, weather effects or background motion, often resulting in frequent false alarms [2].

More recent CCTV systems incorporate AI-based features that offer more reliable analysis than traditional motion detection. These capabilities are typically built into Network Video Recorders (NVRs), which are devices that store footage and perform on-device video processing. Because NVRs must handle multiple video streams at once and operate with limited computing power, they use small, specialised models designed to run efficiently on embedded hardware. These models can support tasks such as person or vehicle detection, but they are significantly less complex than state-of-the-art research models such as the YOLO family. Running larger models requires dedicated GPUs and far more processing power, making them impractical for most commercial CCTV deployments. NVRs with enhanced AI features also tend to be more expensive, yet still cannot match the accuracy or capabilities of larger vision models.

These limitations highlight the ongoing need for efficient, lightweight computer vision methods

that can operate reliably on constrained hardware, particularly in real-time surveillance settings.

2.3 Synthetic Data

Synthetic data refers to data generated by computer systems rather than captured from the real world, and in computer vision this typically includes synthetic images created using generative models [18,19]. Recent advances in generative AI have made it possible to create large collections of images that resemble real-world scenes. These systems can generate images either from existing examples or entirely from text descriptions, allowing users to control factors such as lighting, weather, viewpoint and scene layout. This level of control makes it possible to produce training data that would be difficult or impractical to collect with real cameras.

Synthetic data is valuable in situations where real images are limited, difficult to obtain or cannot be shared due to privacy constraints. It also allows training datasets to include variations that rarely occur in practice, helping models become more robust to unfamiliar conditions. A common challenge is the domain gap, which refers to visual differences between synthetic and real images that can reduce performance when models trained mainly on synthetic examples are applied to real footage [20].

However, research shows that this gap is not always a limiting factor. When synthetic images are sufficiently realistic, or when synthetic and real datasets are combined during training, models can achieve performance comparable to or better than training with real data alone [20]. As generative methods continue to improve, synthetic data is increasingly viewed as a practical and flexible way to expand training datasets and support a wide range of computer vision tasks.

2.4 Edge Computing

Edge computing places processing resources close to where data is generated, reducing the need to send information to distant cloud servers for analysis [7, 8]. It emerged as an alternative to cloud-only systems, which introduce latency, increase bandwidth usage, and rely on stable network connectivity over long distances. Edge devices typically operate with limited processing power and memory, which restricts the size of the models they can support in real time. Devices such as the Raspberry Pi or Jetson Nano are common examples of low-power edge hardware that can run AI models locally, but only within these computational constraints.

Research on edge-based video analytics shows that performing computation near the camera reduces communication delays and lowers the amount of data that needs to be transmitted [8]. Additional work reports that keeping processing within the local network improves reliability and avoids issues caused by external communication failures [4]. These findings show that edge computing can support real-time video analytics in environments with constrained or inconsistent network conditions [5].

2.4.1 Measuring Inference Speed on Edge Devices

On edge devices, how quickly a model can process an image is as important as how accurately it detects objects. Two related measurements are commonly used to describe inference speed.

Latency is the time the model takes to process a single image, usually reported in milliseconds. *Throughput* is the number of images the model can process per second, usually reported as frames per second (FPS). The two are directly related: if latency is measured in milliseconds, throughput in frames per second can be calculated as

$$\text{FPS} = \frac{1000}{\text{Latency (ms)}}.$$

Lower latency and higher FPS both indicate a faster model. For real-time video analysis such as CCTV surveillance, inference speed must be high enough to keep pace with the camera stream, even if it is not as fast as the full frame rate of the camera.

2.4.2 Quantization for Edge Deployment

Deep learning models store the numbers they use for calculations in a format called FP32 (32-bit floating point), which provides high precision but requires a large amount of memory. Quantization reduces model size by converting these values to a simpler format such as INT8 (8-bit integer), which uses only 8 bits per value instead of 32 [16]. Because INT8 values are four times smaller, the overall model file becomes significantly smaller and the calculations run faster, particularly on devices that support integer arithmetic efficiently. The trade-off is that INT8 has fewer possible values than FP32, so some precision is lost when rounding the original numbers to fit the simpler format.

The most common approach for applying quantization after a model has already been trained is called post-training quantization (PTQ). In PTQ, a small set of representative images, known as calibration data, is passed through the trained model to observe the range of values that each layer produces. These observed ranges are then used to determine how to map the original FP32 values into the INT8 range as accurately as possible. This process does not require retraining the model, which makes it straightforward to apply to any existing trained model.

Quantization is widely used in real-time vision systems where fast processing is more important than maintaining the highest possible accuracy. In practice, INT8 quantization can substantially reduce model size and improve inference speed while keeping accuracy close to the original full-precision model. For example, Boddu [17] reports that quantizing a YOLOv4-Tiny detector reduces its size from 22.5 MB to 6.4 MB (a 71% reduction) while achieving comparable detection performance to the full-precision baseline.

2.5 Relevance to This Study

The topics in this section provide the background needed for the experiments in this project. Object detection describes the core vision task used in CCTV analytics. Its training procedures and evaluation metrics explain how model performance is measured.

Synthetic data introduces a controlled way to generate training images when real CCTV footage is limited. The domain gap highlights why performance differences between synthetic and real

images must be examined.

Edge computing and quantization outline the practical constraints of running models on low-power hardware. These constraints motivate the evaluation of lightweight and quantized models on the Raspberry Pi 5.

3 Methodology

This section describes the dataset, hardware, and training configuration used across all experiments. Individual experiment sections describe only the aspects specific to each experiment.

3.1 Dataset Construction

This subsection outlines the steps used to collect, annotate, clean, and organise the real and synthetic datasets for use in the experiments.

Data Collection and Annotation. Two datasets are used in this study: a real CCTV dataset and a synthetic dataset generated using a generative AI model.

Real CCTV Data. The real dataset is collected from three sources: Creative Commons CCTV footage obtained from YouTube, footage provided by a company for research purposes, and recordings from privately owned home cameras. For the YouTube videos, the PyTube Python library is used to download the footage, and OpenCV extracts individual frames from the video files. The other two sources already provide individual images, so no frame extraction is required. All real images are reviewed to remove blurred frames, duplicates, and frames that do not contain people.

Synthetic CCTV Data. The synthetic dataset is created using the Gemini 2.5 Flash generative model [33]. Empty CCTV backgrounds are first prepared by removing people from selected real frames, allowing the model to produce clean background images. These backgrounds are then used as inputs for fully synthetic generation: the model creates new CCTV-style scenes containing synthetic people placed within variations of these backgrounds. All generated images are therefore fully synthetic, with a mixture of day and night scenes produced to reflect the diversity of the real footage.

Annotation and Preprocessing. Both datasets follow the same annotation and cleaning process to ensure consistency across all experiments. Every image is annotated in CVAT [34], an open-source image annotation tool that allows users to draw bounding boxes around objects of interest in each image. A single class label, person, is used because the goal of this study is person detection. CVAT automatically generates a bounding box for each visible person in an image, and each box is linked to the person label. Every annotation contains two components: the class name and four bounding-box coordinates that describe the location of the person in the image.

Dataset Summary. Table 1 summarises the total number of images collected from each source and the generated synthetic datasets.

Table 1: Summary of real and synthetic datasets used in the study.

Dataset Source	**Number of Images**
YouTube (Creative Commons CCTV)	1,637
Home CCTV Footage	11,772
Company CCTV Footage	8,807
Total Real Images	**22,216**
Synthetic (Fully Synthetic Scenes)	10,216
Total Synthetic Images	**10,216**

Duplicate Removal. Before preparing the dataset for training, a complete duplicate-removal stage was carried out. CCTV footage is captured continuously, often recording frames only milliseconds apart. As a result, long sequences of frames contain almost no meaningful change from one image to the next. If all of these frames are kept, the model is repeatedly shown nearly identical scenes instead of learning from a wider range of examples.

Large numbers of near-identical frames create a dataset that appears large but is effectively narrow. Although more than twenty thousand images were collected, many of them capture almost the same scene: the same background, the same lighting and the same person in nearly the same pose. These frames repeat the same visual pattern rather than expanding the diversity of training examples. When a model trains on such narrow variation, it becomes good at recognising that one specific scene but performs poorly when the camera angle, lighting or person appearance changes. Removing duplicates is therefore essential to ensure that the training data actually represents the range of situations encountered in real CCTV footage.

To detect and remove near-identical frames, each real image was converted into a numerical representation called an embedding. An embedding is a vector, meaning an ordered list of numerical values generated by a vision model [35]. These values summarise the visual content of the image in a consistent format. Images that look similar produce vectors with similar numerical patterns, while images that look different produce vectors that differ. This allows similarity to be measured reliably without comparing pixel values directly.

Embeddings were generated using the "Self-Supervised Descriptor for Image Copy Detection" (SSCD) model [36]. SSCD was selected because it is designed specifically for identifying images that look alike, even when the differences between them are very small. This makes it well suited for CCTV footage, where many frames differ only slightly due to the camera capturing motion in very small steps.

Once embeddings were generated, image similarity was measured using cosine similarity [37]. Each embedding is a vector, and each number in the vector represents a learned feature of the image. Because the vector contains many values, it can be treated as a point in a space where each value corresponds to one coordinate. Images that look similar produce vectors that fall in

Figure 1: Examples of near-duplicate CCTV frames. These frames were captured milliseconds apart and contain almost the same visual content.

similar positions within this space. Cosine similarity measures how closely two vectors point in the same direction. When the directions are very similar, the score is close to 1, indicating that the images contain nearly the same visual content. Lower scores indicate greater differences.

All 22,216 real images were compared using this method. The process began by selecting one image as the reference frame. Every other image was compared to this reference, and any image with a cosine similarity of at least 0.90 was placed into the same group. Once a group was formed, all of its images were removed from consideration. The next ungrouped image was then selected as the new reference frame, and the same steps were repeated until no images remained.

This procedure ensures that each group is built around a single clearly defined reference image. It also prevents a chaining effect. In a chaining effect, an image could be grouped simply because it resembles another image within the group. In this approach, every image must independently meet the 0.90 similarity threshold with the reference frame for that group. This guarantees that each group contains frames that are genuinely near-identical and not loosely linked through intermediate similarities.

After duplicate removal, the real dataset was reduced to 4,519 unique images. Figure 2 shows examples of these remaining frames, each representing a different scene or viewpoint present in the dataset.

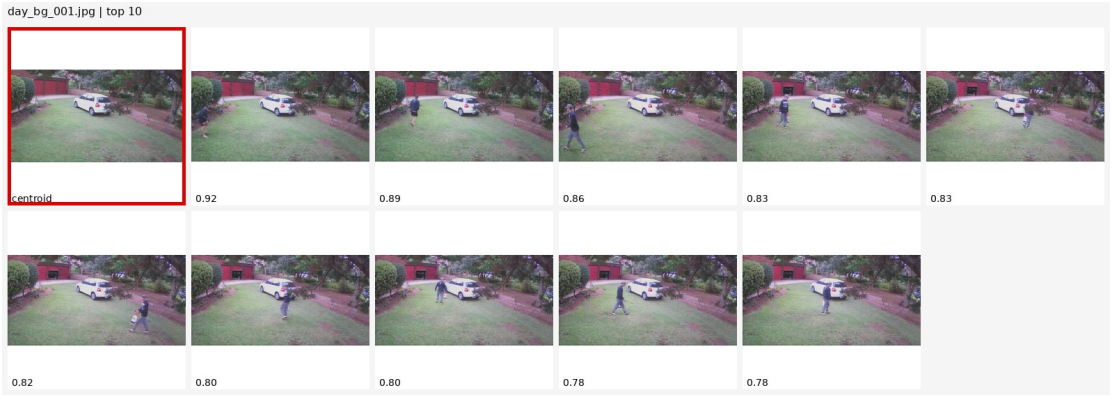

Figure 2: Examples of unique CCTV frames after duplicate removal. Each image captures a different scene and contributes new information to the dataset.

Table 2 summarises the reduction in dataset size resulting from this duplication-removal process.

Table 2: Real dataset size before and after duplicate removal.

Stage	**Number of Images**
Real Images (Before Deduplication)	22,216
Real Images (After Deduplication)	4,519

Grouping Images by Scene. After duplicate removal, the next step was to organise the real images into groups based on the type of scene they belonged to. CCTV datasets often contain footage recorded from several different camera positions, each with its own background, angle and layout. To handle this structure properly, the deduplicated real images were grouped according to these distinct camera viewpoints.

To form these groups, a set of 30 representative background images was selected by manually reviewing the deduplicated dataset. Each selected image captured a unique camera viewpoint or backdrop and served as a "scene anchor". These anchors acted as reference points that the remaining images could be compared against.

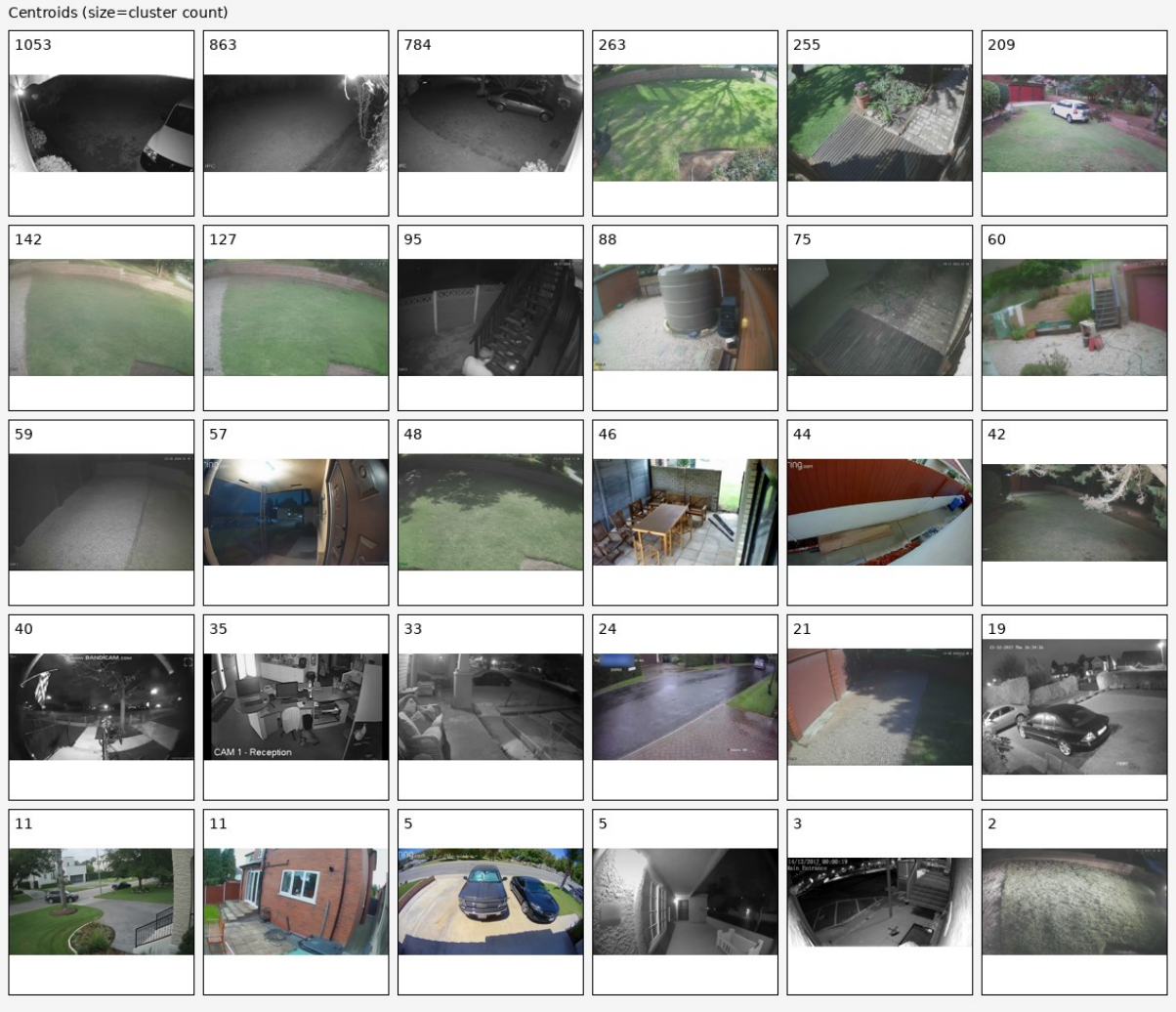

Figure 3: The 30 manually selected scene anchors used to group real images by camera viewpoint. The number above each image indicates the number of real frames assigned to that scene.

Each deduplicated real image was compared with all 30 anchors using the same embedding representations described earlier. For every image, the anchor with the highest cosine similarity score was identified, meaning it was the most visually similar. The image was then assigned to the group of that anchor, effectively placing it into the scene category it best matched.

Every deduplicated image was placed into the single scene group it most closely matched. These groups organise the dataset into clear camera viewpoints, which is essential for the later stages of dataset preparation.

Data Splitting

This section describes how the processed datasets were divided and structured to ensure fair, unbiased, and leakage-free evaluation across all experiments.

Train–Validation–Test Splitting. A train–validation–test split divides the dataset into three separate parts, each serving a different stage of model development. This structure ensures

that the model is always evaluated on images it has not seen before. The training set is used to teach the model to recognise people, the validation set is used to check whether the model is learning general patterns rather than memorising the training images, and the test set is kept completely separate until the end. Testing on this final set provides an unbiased measure of how well the model performs on genuinely new CCTV scenes.

The training set contains most of the images. As the model trains, it gradually improves by adjusting the internal values it uses to make predictions. The validation set contains different images that are not used for training and shows whether the model can generalise rather than memorise. Memorisation leads to overfitting, which causes good performance on familiar images but poor performance on new ones. The test set is held back until the end of the project and is used only once to obtain a final performance measurement.

A target split of 70% for training, 10% for validation and 20% for testing was used as guidance. Each image was placed into exactly one of these sets, and similar or near-duplicate images were not allowed to appear in different splits, as this would artificially inflate performance and give an unrealistic impression of model accuracy.

Camera-View Separation (Preventing Data Leakage). After duplicate removal and scene grouping, each real image belonged to one of 30 camera views, where each view represented a distinct CCTV background or perspective. If the same camera view appeared across multiple splits, the model could recognise the background itself rather than learning to detect people, causing data leakage. To prevent this, every camera view was assigned entirely to a single split, and no view appeared in more than one of the training, validation or test sets.

This ensures that the model must detect people in genuinely new environments. Although keeping each camera view intact makes it harder to match the exact 70/10/20 split, it produces a far more reliable and meaningful evaluation.

Handling Imbalanced Scene Sizes. The real dataset contained large differences in how many images came from each camera view. Most views contained between 20 and 260 images, but three nighttime views were exceptionally large, containing 1,053, 863 and 784 images. Together, these three views accounted for 2,700 images, or 59.7% of the entire deduplicated dataset.

If all images from these large views were kept intact and placed into a single split, that split would become dominated by a single camera perspective. To avoid this imbalance, the three largest nighttime views were reduced before splitting. Their sizes were capped to 500, 400 and 400 images. This preserved important nighttime scenes while preventing any one view from overwhelming a split.

After this adjustment, the dataset decreased from 4,519 to 3,119 images. All 27 smaller camera views were kept in full, and each of the three capped views was still assigned entirely to one split to maintain strict separation between environments.

Table 3: Sizes of the three largest nighttime camera views before and after capping.

Camera View	Original Size	Capped Size
night_bg_003	1,053	500
night_bg_002	863	400
night_bg_005	784	400
Total	**2,700**	**1,300**

Day–Night Balancing. The dataset contained far more nighttime images than daytime images. Without balancing, one or more splits could easily become heavily night-dominant. To avoid this, daytime and nighttime camera views were examined separately and distributed so that each split contained a mixture of both conditions.

After splitting, the training set contained approximately 46% daytime and 54% nighttime images. The validation and test sets each contained around 40% daytime and 60% nighttime images. These proportions provide a realistic but balanced foundation for evaluating the model while still ensuring that each camera view appears in only one split.

Synthetic Data Splitting. Synthetic images were split using the same principles as the real data to ensure a fair comparison. The synthetic dataset contained 10,216 images generated from 100 unique backgrounds. Images were grouped by background, and each background was assigned entirely to either the training or validation set. Selected synthetic backgrounds contained more images than needed, so a small number were reduced in size to ensure that the synthetic training and validation sets matched the real-data sizes exactly: 1,795 training images and 664 validation images. Both the real-trained and synthetic-trained models were evaluated on the same 660-image real test set, ensuring a fair and unbiased comparison.

Together, these steps produced train–validation–test splits that are balanced, free from data leakage and suitable for evaluating the effect of training on real versus synthetic CCTV data.

Final Dataset Composition. Table 4 summarises the final train–validation–test splits for both the real and synthetic datasets.

Table 4: Final split sizes for the real and synthetic datasets.

Split	Real Images	Synthetic Images
Training	1,795	1,795
Validation	664	664
Test	660	–
Total	**3,119**	**2,459**

3.2 Hardware and Training Configuration

All training was carried out on a cloud GPU server rented through Vast.ai, using an NVIDIA RTX 4090 (24 GB VRAM). This provided sufficient computational capacity to train with high-

resolution images and ensured that all models used the same hardware configuration.

The Ultralytics framework was used to load pretrained models, fine-tune them on each dataset, and evaluate performance using standard object-detection metrics. Using the same framework across all training runs ensured that the process remained consistent, reliable, and fully reproducible.

The following training parameters were kept constant across all experiments:

- **Image size:** 832 px input resolution, chosen to preserve detail needed for small or partially visible people in CCTV scenes.
- **Batch size:** 16 images per update step, which fit comfortably within the RTX 4090 memory while providing stable optimisation.
- **Learning rate:** 2×10^{-4}, selected to provide steady learning without causing unstable behaviour on the relatively small datasets.
- **Epochs:** 40 full passes through the training data, which preliminary tests showed to be sufficient for convergence.
- **Backbone:** unfrozen. The backbone refers to the main feature-extracting layers of the model, and allowing them to update helps the model adapt from COCO images to CCTV footage.

4 Experiments and Results

For each experiment, the specific method is described first, followed by the results and a brief analysis of the key findings.

4.1 Experiment 1: Real vs Synthetic Training Data

This experiment investigates RQ1 by comparing person detection models trained on real and synthetic CCTV data.

Objectives:

- Create two training conditions: one using real CCTV images and one using fully synthetic CCTV-style images.
- Train identical person detection models under both conditions to ensure a fair comparison.
- Evaluate each model using mAP, precision, and recall.
- Compare performance across both datasets to assess whether synthetic data can match or exceed the accuracy achieved with real CCTV footage.
- Analyse the strengths and weaknesses of synthetic data by examining common error patterns, such as missed detections or false positives.

4.1.1 Method

Three models were evaluated in this experiment. The first is the YOLO11n model pretrained on the COCO dataset, evaluated without any fine-tuning on CCTV data, which serves as a baseline reference. The second is a YOLO11n model fine-tuned on the 1,795-image real CCTV training set, and the third is a YOLO11n model fine-tuned on the 1,795-image synthetic training set, as described in Section 3.1 (Table 4). Apart from the training data, all other aspects of the training process were kept constant using the shared configuration described in Section 3.2, so that any differences in performance could be attributed solely to the type of data used. All three models were evaluated on the same 660-image real CCTV test set using the metrics defined in Section 2.1.1: mAP@0.50, mAP@0.50:0.95, precision, and recall.

4.1.2 Results

The following results compare all three models on the 660-image real CCTV test set (Section 3.1, Table 4).

Overall Model Comparison. Table 5 summarises the test-set performance of all three models. The best result in each metric is highlighted in bold. The real-trained model achieves the highest scores across every metric, improving notably over the pretrained baseline. The synthetic-trained model performs substantially worse on real footage, indicating limited transfer from synthetic scenes to real environments.

Table 5: Final test metrics for the baseline, real-trained, and synthetic-trained models on the real CCTV test set.

Model	**mAP@0.50**	**mAP@0.50:0.95**	**Precision**	**Recall**
Baseline (Pretrained)	0.853	0.671	0.922	0.753
Real-Trained	**0.931**	**0.735**	**0.949**	**0.893**
Synthetic-Trained	0.719	0.503	0.769	0.615

These differences are visualised in Figure 4. The real-trained model achieves the largest improvements in both mAP and recall, demonstrating strong adaptation to CCTV scenes. The synthetic-trained model consistently underperforms, particularly in recall, indicating that many people in the test images were missed.

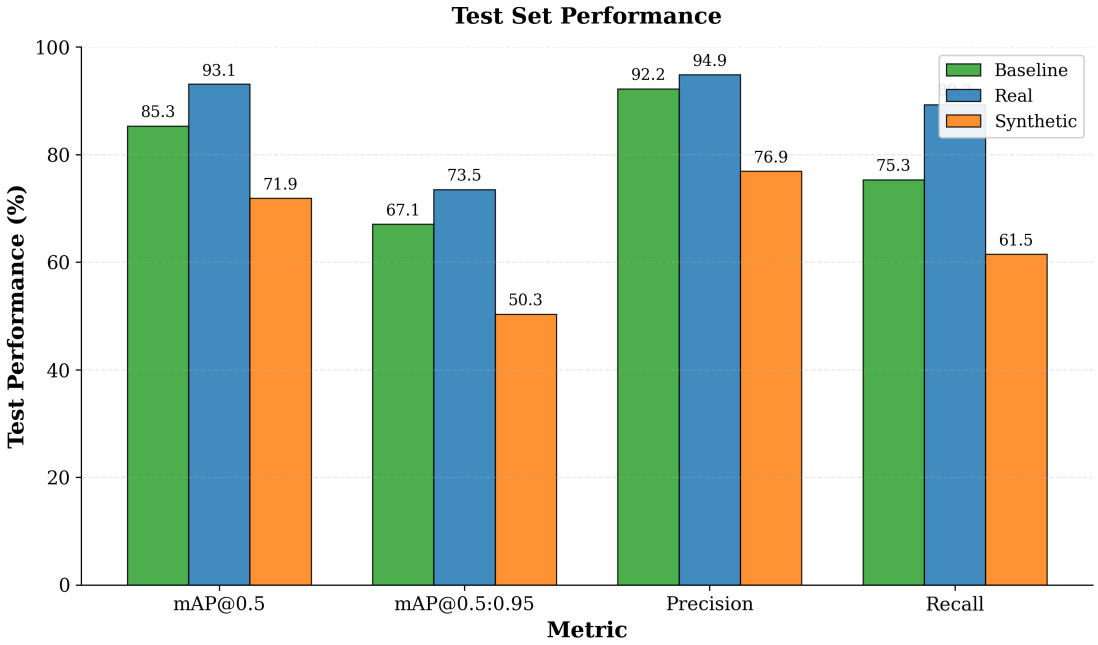

Figure 4: Comparison of the three models on the real CCTV test set across all evaluation metrics.

Training Behaviour. Figure 5 shows the mAP@0.50:0.95 curves recorded during training. The synthetic-trained model reaches high validation accuracy early but does not translate this into strong test performance. The real-trained model improves more steadily across the 40 epochs and ultimately yields the highest performance on the real test images.

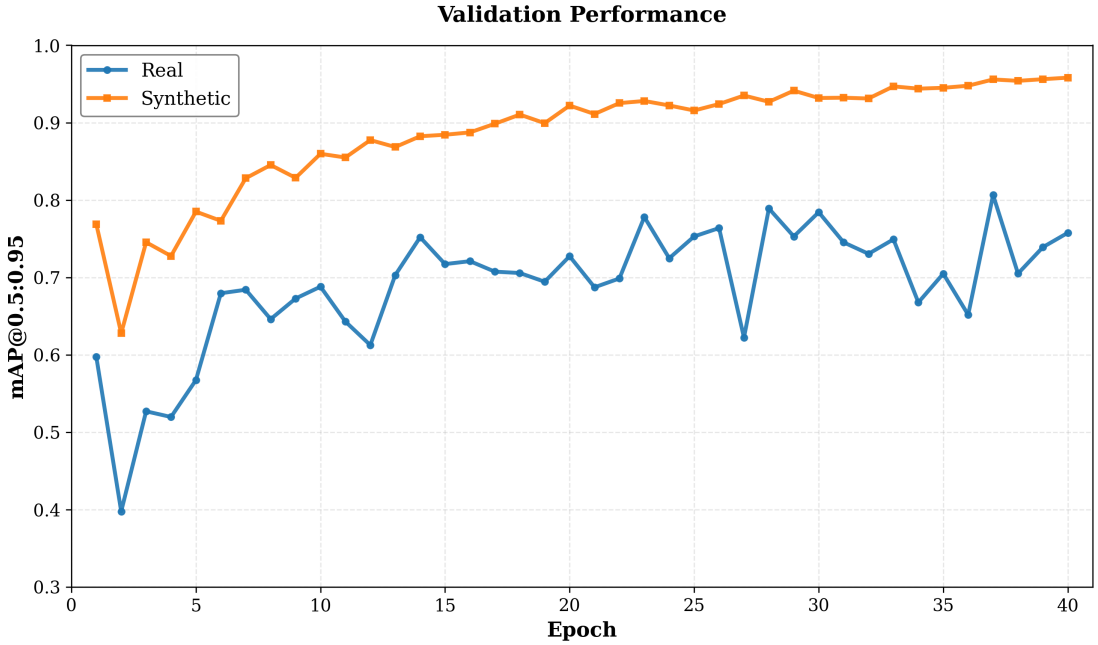

Figure 5: Training curves showing mAP@0.50:0.95 across 40 epochs for the real-trained and synthetic-trained models.

Error Analysis. A closer inspection of the confusion matrices (Figure 6) highlights clear behavioural differences between the two models. Each matrix shows the number of predictions made by the model (rows) against the actual ground-truth labels (columns). A prediction counted under "Person / Person" is a correct detection (true positive), while "Background / Person" indicates a person that the model failed to detect (false negative), and "Person / Background" indicates a detection where no person was actually present (false positive). The real-trained model correctly detected 711 person instances (89.7%) while missing only 82, and it produced just 38 false positives. In contrast, the synthetic-trained model detected 554 true positives (69.9%) and missed 239 people, which is more than triple the real-trained model's misses. It also produced 305 false positives, indicating a strong tendency to interpret background regions as people. These patterns match the overall results: the synthetic-trained model shows much lower recall and precision, demonstrating that training solely on synthetic CCTV-style data does not generalise well to real scenes.

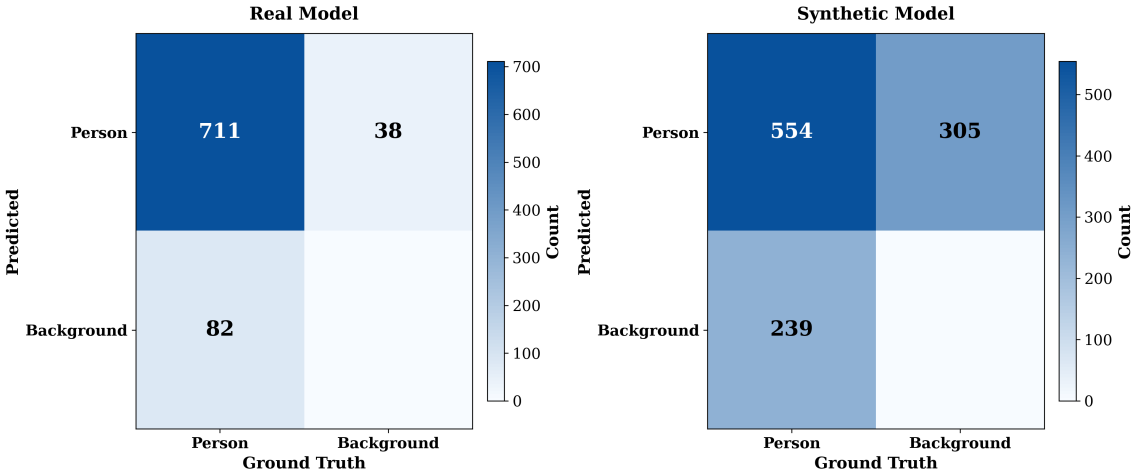

Figure 6: Confusion matrices for the real-trained and synthetic-trained models on the test set. Rows represent model predictions and columns represent ground-truth labels.

Summary. Across all metrics, the real-trained model achieves the strongest performance. Fine-tuning on real CCTV images provides a clear benefit over the pretrained baseline, particularly in recall and overall detection quality. The synthetic-trained model, despite learning quickly during training, performs poorly on real footage. This result is consistent with Vanherle et al. [20], who found that the domain gap between synthetic and real images is a key factor limiting detection performance. However, it contrasts with Fabbri et al. [23], who achieved strong pedestrian detection using only synthetic data rendered from a video game engine. Their dataset offered far greater visual diversity and was designed for street-level pedestrian scenes rather than overhead CCTV footage, which likely reduced the domain gap. In this study, the synthetic images were generated from a limited set of CCTV backgrounds using a generative model, producing a larger gap between training and test conditions. These findings suggest that synthetic data produced by current generative models does not yet generalise well enough to replace real CCTV footage for person detection.

4.2 Experiment 2: Edge-Optimised Architecture Comparison

To investigate RQ2, this experiment compares four edge-optimised object detection architectures for person detection on real CCTV footage.

Objectives:

- Select a set of edge-optimised detection models spanning both CNN and transformer architectures.
- Train all models on the same real CCTV dataset using identical hyperparameters where possible.
- Evaluate each model on the same held-out real test set using mAP, precision, and recall.
- Compare the results to determine which architecture achieves the best detection accuracy for this task.

4.2.1 Method

Four nano-class models were selected for this experiment, each representing a recent edge-optimised architecture for real-time object detection. Three are CNN-based models from the YOLO family: YOLO11n [30], YOLOv10n [38], and YOLO26n [39]. These were chosen because they are the most recent nano-scale YOLO variants available at the time of this study, each designed for low-latency inference on resource-constrained hardware. The fourth model is RF-DETR-N [40], a transformer-based detector from Roboflow, included to compare the CNN-based YOLO approach against a transformer-based alternative on this task. Table 6 summarises the key characteristics of each model.

Table 6: Models compared in Experiment 2.

Model	**Type**	**Framework**	**Parameters**
YOLO11n	CNN	Ultralytics	2.6M
YOLOv10n	CNN (NMS-free)	Ultralytics	2.3M
YOLO26n	CNN (NMS-free)	Ultralytics	2.4M
RF-DETR-N	Transformer	Roboflow	30.1M

All three Ultralytics models were trained using the shared configuration described in Section 3.2. RF-DETR-N required a separate training pipeline because it uses the `rfdetr` package and expects COCO JSON annotations rather than YOLO-format labels. The YOLO-format annotations were converted to COCO JSON format before training. RF-DETR-N was trained with a batch size of 4 and gradient accumulation over 4 steps, giving an effective batch size of 16 to match the other models. Its input resolution was set to 896 pixels, the closest value to 832 that is divisible by 56, which is a requirement of the RF-DETR architecture. All other settings, including the number of epochs (40) and learning rate (2×10^{-4}), were kept the same.

All four models were evaluated on the same 660-image real CCTV test set (Section 3.1, Table 4) using the metrics defined in Section 2.1.1: mAP@0.50, mAP@0.50:0.95, precision, and recall.

4.2.2 Results

Table 7 presents the test-set performance of all four models on the real CCTV test set.

Table 7: Test metrics for all four architectures on the real CCTV test set.

Model	mAP@0.50	mAP@0.50:0.95	Precision	Recall
YOLO11n	0.931	0.735	0.949	0.893
YOLOv10n	**0.935**	0.702	**0.960**	0.859
YOLO26n	0.924	**0.745**	0.931	**0.884**
RF-DETR-N	0.916	0.724	–	0.770

These results are visualised in Figure 7. All three YOLO models perform closely across the main metrics, while RF-DETR-N falls behind, particularly in recall.

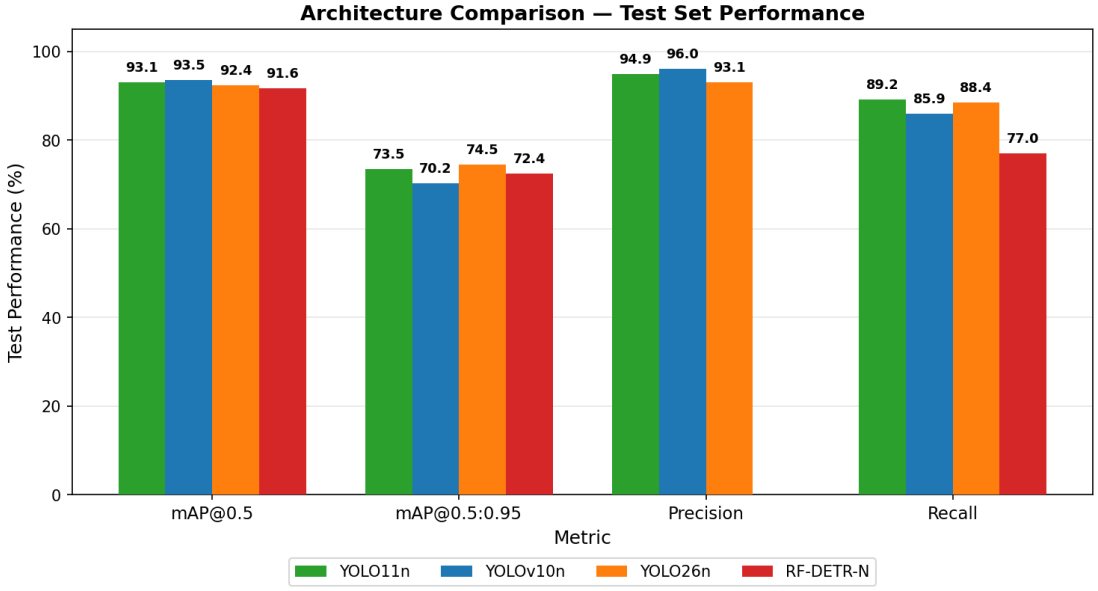

Figure 7: Comparison of the four architectures on the real CCTV test set across all evaluation metrics.

YOLO Model Comparison. Among the three YOLO models, performance is closely matched. YOLOv10n achieves the highest mAP@0.50 (0.935) and precision (0.960), meaning it produces the fewest false positives. YOLO26n achieves the highest mAP@0.50:0.95 (0.745), indicating the most accurate bounding box localisation across strict IoU thresholds. YOLO11n achieves the highest recall (0.893), meaning it misses the fewest people. The differences between the three YOLO models are small, with all achieving mAP@0.50 above 0.92 and recall above 0.85.

RF-DETR-N Performance. RF-DETR-N scored the lowest across all models. Its recall of 0.770 is notably lower than the YOLO models, meaning it missed a larger proportion of people in the test images. Several factors contribute to this result. RF-DETR-N has 30.1 million

parameters, which is roughly 12 times more than any of the YOLO models. With only 1,795 training images, the model is heavily over-parameterised for this task. Transformer architectures are known to require larger datasets to train effectively, and single-class person detection on fixed-angle CCTV footage does not benefit from the global context modelling that transformers are designed for. Additionally, RF-DETR-N applies only simple image flipping during training, whereas the Ultralytics YOLO models apply a much richer set of image augmentations, which expose the model to more visual variety during training.

Summary. All three YOLO architectures achieve strong and closely matched performance on the real CCTV test set. YOLO26n achieves the best overall localisation quality (mAP@0.50:0.95), YOLOv10n achieves the highest precision, and YOLO11n achieves the highest recall. RF-DETR-N underperforms on this task, particularly in recall, despite having roughly 12 times more parameters than any of the YOLO models. This is consistent with research showing that transformer-based models need much larger training datasets than CNN-based models to learn effectively, because CNNs are designed to detect local patterns such as edges and shapes, which makes them effective even with limited data, whereas transformers must learn these patterns entirely from the training examples provided [29]. Without enough training data, transformers tend to memorise the examples they have seen rather than learning general patterns, which reduces their ability to detect objects in new images [41]. The original DETR transformer detector was trained on over 118,000 images [42], whereas this experiment used only 1,795, which is far too few for a model of this size to learn reliably. Although recent studies have shown that transformer detectors can outperform YOLO models on large benchmarks [43], these results show that the advantage does not hold on small, focused datasets like the one used in this study, where lightweight CNN-based models remain the stronger choice.

4.3 Experiment 3: Quantization and Edge Deployment

To investigate RQ3, this experiment evaluates how model accuracy and inference speed change when trained models are exported to ONNX format and quantized to INT8 for deployment on the Raspberry Pi 5.

Objectives:

- Export each trained model from PyTorch to FP32 ONNX format and verify that export does not introduce significant accuracy loss.
- Apply INT8 post-training quantization and measure the resulting change in mAP, precision, and recall.
- Record the model file sizes before and after quantization to measure compression.
- Benchmark inference latency and throughput (FPS) for each model on the Raspberry Pi 5.
- Identify which model achieves the best balance of accuracy and speed for edge deployment.

4.3.1 Method

The three YOLO models from Experiment 2 (YOLO11n, YOLOv10n, and YOLO26n) were taken through a two-stage pipeline: PyTorch to ONNX export, producing the Original model (FP32, 32-bit), and then quantization, producing the Quantized model (INT8, 8-bit). These terms, Original and Quantized, are used throughout the tables and figures in this section. The quantized models were then tested for inference speed on the Raspberry Pi 5. RF-DETR-N was excluded from this experiment because transformer-based models are too slow to run on the Raspberry Pi 5 CPU, making edge deployment impractical. Figure 8 illustrates this pipeline.

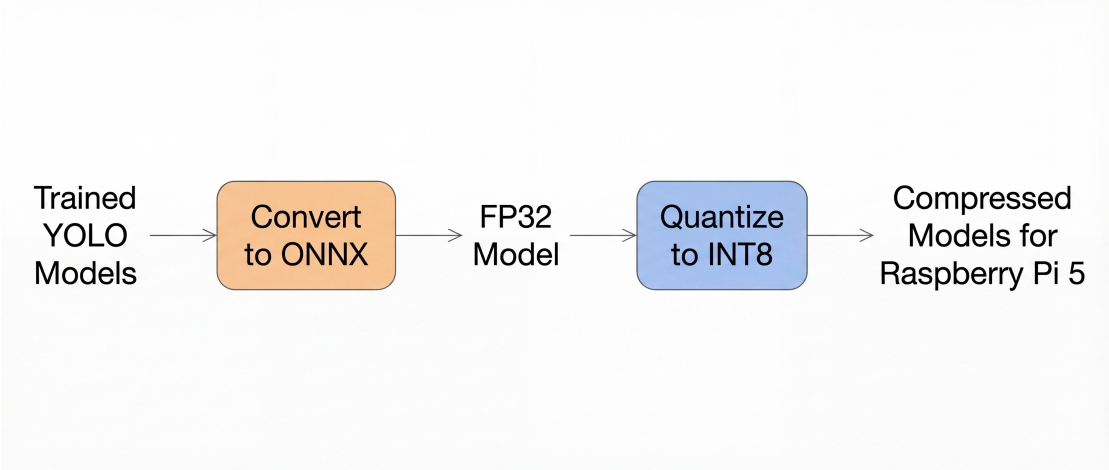

Figure 8: Export and quantization pipeline used in Experiment 3.

The three models trained on the 1,795-image real CCTV training set in Experiment 2 were each exported from PyTorch to ONNX format using the Ultralytics export function. The Original models were then evaluated on the 660-image real CCTV test set (Section 3.1, Table 4) to verify that export did not introduce significant accuracy loss.

INT8 quantization was applied to each model using ONNX Runtime's static quantization method. A calibration set of 100 images, selected randomly from the 1,795-image real CCTV training set (Section 3.1, Table 4), was used to determine the appropriate value ranges for each layer, following the calibration process described in Section 2.4.2. The detection head layers were excluded from quantization and kept in their original 32-bit format, as quantizing these layers produced zero detections in initial tests. This mixed approach, where the feature-extraction layers are quantized but the detection head remains in its original format, is standard practice for object detection quantization.

To measure how fast each model processes images on real edge hardware, each model was run on a Raspberry Pi 5 using ONNX Runtime. Each model was tested on 200 real CCTV images at 832×832 resolution, with 10 warmup runs before timing to ensure stable measurements. The time taken to process each image (latency) was recorded and averaged across all 200 images to calculate throughput in frames per second (FPS).

4.3.2 Results

Accuracy Through the Export and Quantization Pipeline. Table 8 shows how accuracy changes at each stage of the pipeline illustrated in Figure 8, from the original PyTorch model through export to quantization.

Table 8: mAP at each stage of the export and quantization pipeline.

Model	Metric	PyTorch	Original	Quantized
YOLO11n	mAP@0.50	0.931	0.921	0.906
	mAP@0.50:0.95	0.735	0.723	0.699
YOLOv10n	mAP@0.50	0.935	0.926	0.917
	mAP@0.50:0.95	0.702	0.688	0.700
YOLO26n	mAP@0.50	0.924	0.923	0.907
	mAP@0.50:0.95	0.745	0.742	0.712

The export stage introduced small accuracy drops of 0.1–1.3% mAP, which is expected when converting between frameworks due to differences in floating-point rounding, operator implementations, and graph optimisations. YOLO26n showed the smallest export drop (0.1% mAP@0.50), which is consistent with its architecture being specifically designed for efficient model export [39].

Quantization reduced accuracy by a further 1–3% mAP@0.50:0.95 for YOLO11n and YOLO26n. YOLOv10n showed an unusual result: its Quantized accuracy (0.70 mAP@0.50:0.95) was slightly higher than its Original value (0.69). Recent research has shown that quantization does not always reduce accuracy, and that in some cases it can improve performance by forcing the model to rely on more general patterns rather than overfitting to specific details in the training data [44]. This effect was not observed in the other two models, where the overall accuracy loss from quantization was larger.

Model Size Reduction. Table 9 shows the file sizes of the Original and Quantized ONNX models.

Table 9: ONNX model sizes before and after quantization.

Model	Original (MB)	Quantized (MB)	Reduction
YOLO11n	10.2	5.4	47%
YOLOv10n	9.0	4.8	47%
YOLO26n	9.5	4.6	52%

All three models achieved approximately 47–52% size reduction through quantization. YOLO26n achieved the largest reduction (52%) and the smallest Quantized model size (4.6 MB), making it the most compact option for edge deployment.

Raspberry Pi 5 Inference Performance. Table 10 presents the inference latency and throughput measured on the Raspberry Pi 5.

Table 10: Inference performance on Raspberry Pi 5 (ONNX Runtime, 832×832 input).

Model	**Format**	**Avg (ms)**	**Median (ms)**	**FPS**	**Speedup**
YOLO11n	Original	328.7	321.8	3.04	–
YOLO11n	Quantized	207.5	205.5	4.82	1.58×
YOLOv10n	Original	385.8	388.6	2.59	–
YOLOv10n	Quantized	246.8	247.4	4.05	1.56×
YOLO26n	Original	333.9	346.8	3.00	–
YOLO26n	Quantized	**188.1**	**199.3**	**5.32**	1.78×

Quantization improved inference speed by 1.56–1.78× across all models. YOLO26n Quantized achieved the fastest inference at 188.1 ms average (5.32 FPS), which represents a 1.78× speedup over its Original version. This is the largest speedup of any model, and YOLO26n Quantized is also the fastest model overall on the Raspberry Pi 5. YOLOv10n was the slowest in both Original and Quantized formats.

These results are visualised in Figure 9. The x-axis shows the average time taken to process one image (latency in milliseconds), where lower values indicate faster inference. The y-axis shows detection accuracy (mAP@0.50:0.95), where higher values indicate better performance. The ideal position is the top-left corner of the plot, which represents a model that is both fast and accurate. Arrows connect each model's Original version to its Quantized version, showing the direction of change after quantization. In all cases, the arrows point left and slightly down, indicating that quantization makes the models faster at the cost of a small reduction in accuracy.

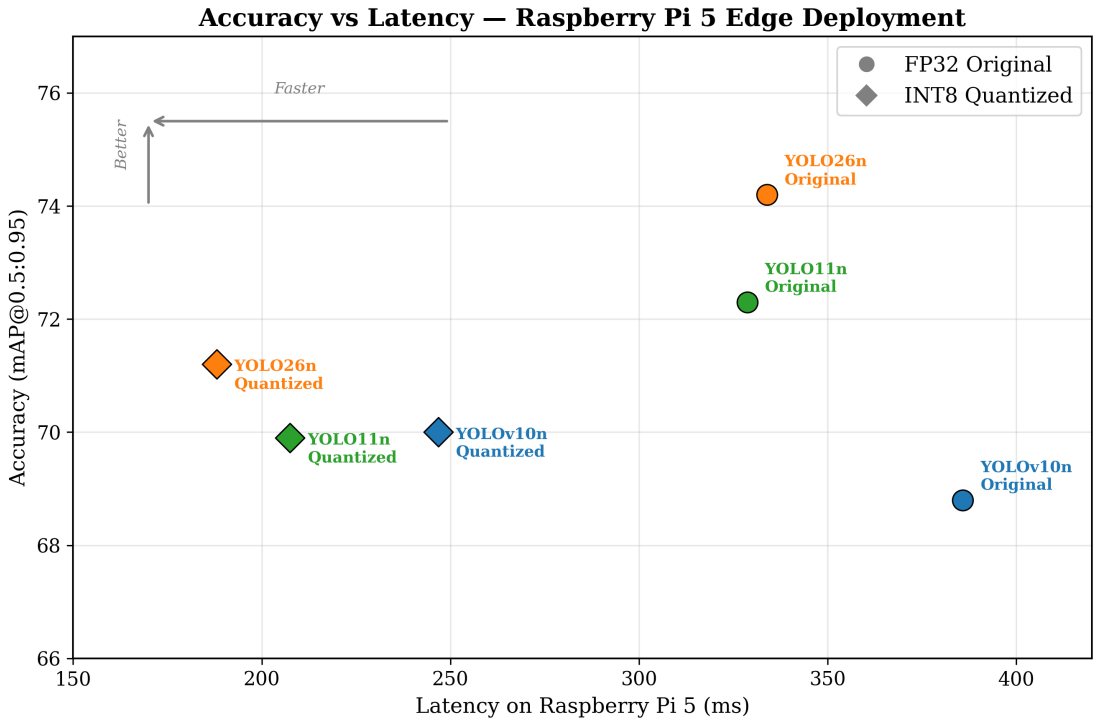

Figure 9: Latency vs accuracy trade-off on Raspberry Pi 5 for all models in Original and Quantized formats.

Summary. YOLO26n Quantized (INT8) emerges as the best model for Raspberry Pi 5 deployment (Figure 9). It achieves the fastest inference speed (5.32 FPS), the smallest model size (4.6 MB), the largest quantization speedup (1.78×), and retains the highest mAP@0.50:0.95 (0.712) among all Quantized models. These results are consistent with previous work on quantized object detection models for edge deployment [17]. The combination of strong accuracy, fast inference, and small size makes YOLO26n the most suitable architecture for edge-based CCTV person detection in this study.

4.4 Experiment 4: Synthetic-to-Real Data Ratio

Experiment 1 showed that training on synthetic data alone does not generalise well to real CCTV footage. However, it is possible that combining a smaller proportion of synthetic data with real data could provide additional visual variation and improve detection performance. To further investigate RQ1, this experiment tests whether synthetic data can serve as a useful supplement to real training data, and at what proportion. YOLO26n was used for this experiment because it achieved the best balance of accuracy and inference speed on the Raspberry Pi 5 in Experiments 2 and 3.

Objectives:

- Create mixed training sets at ratios from 0% to 100% synthetic data, keeping the total number of training images constant.

- Use stratified incremental sampling to ensure consistent day-night composition and that each ratio builds on the previous one.
- Train, export, and quantize a model at each ratio using the same pipeline as Experiment 3.
- Evaluate each Quantized model on the held-out real test set and identify the ratio that achieves the highest accuracy.

4.4.1 Method

For each ratio, a mixed training set was constructed from two sources: the 1,795-image real CCTV training set used in Experiments 1–3, and the synthetic CCTV dataset described in Section 3.1, which contains 5,223 daytime and 4,993 nighttime images. At each ratio, a proportion of real images was replaced with synthetic images, keeping the total training set size fixed at 1,795 images.

To ensure that any change in accuracy between ratios was caused by the quantity of synthetic data and not by variation in which images were selected, a stratified incremental sampling strategy was used. The synthetic pool was split into day and night subsets, and images were drawn proportionally from each subset to maintain a consistent 61%/39% day-night ratio at every ratio. Each ratio was built incrementally on the previous one, meaning the synthetic images at 10% were always a subset of those at 20%, and so on. This ensured that adding more synthetic data always meant adding new images on top of the existing selection, rather than randomly re-sampling a different set.

Eleven ratios were tested: 0%, 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, and 100% synthetic data. For example, at 20% synthetic, the training set contained 1,436 real images and 359 synthetic images (219 day, 140 night).

To account for the randomness involved in which synthetic and real images are selected for each mixed training set, each ratio was trained three times, each time with a different random selection of images. Each model was exported to ONNX and then quantized to INT8 using the same pipeline and hyperparameters described in Experiment 3. All Quantized models were evaluated on the 660-image real CCTV test set (Section 3.1, Table 4) using the metrics defined in Section 2.1.1: mAP@0.50, mAP@0.50:0.95, precision, and recall. Results are reported as the mean and standard deviation across the three runs.

4.4.2 Results

Table 11 presents the mean Quantized (INT8) test accuracy of YOLO26n at each synthetic-to-real ratio, averaged over three runs.

Table 11: YOLO26n Quantized test accuracy at each synthetic-to-real training data ratio (mean ± std over 3 runs).

Synth. %	Real %	mAP@0.50	mAP@0.50:0.95	Precision	Recall
0%	100%	**0.904±0.009**	**0.689±0.015**	**0.926±0.010**	**0.851±0.014**
10%	90%	0.896±0.030	0.691±0.012	0.921±0.004	0.808±0.045
20%	80%	0.855±0.011	0.663±0.009	0.879±0.015	0.768±0.012
30%	70%	0.877±0.015	0.652±0.017	0.907±0.029	0.793±0.011
40%	60%	0.872±0.030	0.660±0.010	0.897±0.017	0.799±0.040
50%	50%	0.833±0.034	0.615±0.033	0.834±0.048	0.736±0.045
60%	40%	0.864±0.041	0.643±0.035	0.886±0.016	0.788±0.051
70%	30%	0.828±0.018	0.634±0.016	0.883±0.013	0.744±0.034
80%	20%	0.755±0.064	0.573±0.050	0.880±0.035	0.639±0.056
90%	10%	0.747±0.051	0.560±0.037	0.832±0.048	0.643±0.059
100%	0%	0.121±0.069	0.085±0.048	0.137±0.082	0.188±0.102

These results are visualised in Figure 10. The x-axis shows the percentage of synthetic data in the training set, and the y-axis shows detection accuracy. The shaded bands represent the standard deviation across the three runs. Accuracy decreases gradually as the proportion of synthetic data increases, with a sharp collapse at 100% synthetic.

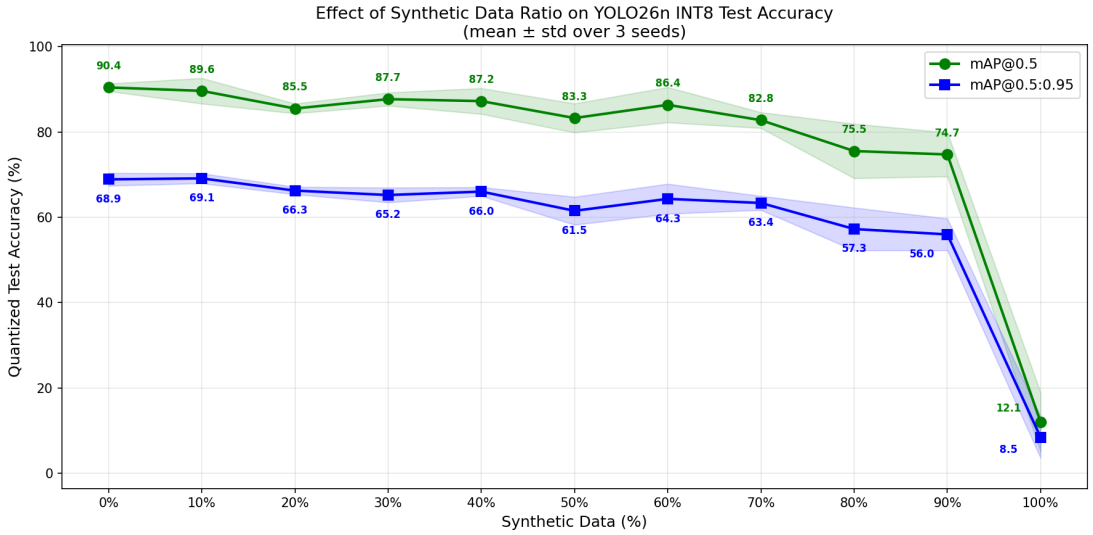

Figure 10: Effect of synthetic data ratio on YOLO26n Quantized test accuracy (mean ± std over 3 runs).

Optimal Ratio. The 0% synthetic ratio (real data only) achieved the strongest overall performance across all metrics (mAP@0.50 of 0.904, precision of 0.926, recall of 0.851). Although 10% synthetic achieved a marginally higher mAP@0.50:0.95 (0.691 vs 0.689), this difference falls within one standard deviation and the 10% ratio scored lower on all other metrics. Be-

yond 10%, accuracy drops more noticeably, indicating that replacing real images with synthetic images reduces performance.

Degradation at High Ratios. Beyond 50% synthetic data, accuracy declines sharply. At 80% synthetic, mAP@0.50:0.95 falls to 0.573, and at 90% it drops to 0.560. The 100% synthetic model collapses entirely (mAP@0.50:0.95 of 0.085), consistent with the findings from Experiment 1 that training solely on synthetic data does not generalise to real CCTV scenes. The domain gap between synthetic and real images becomes the dominant factor when synthetic data makes up the majority of the training set.

Summary. Training on real data alone produces the highest overall accuracy. A small proportion of synthetic data (10%) performs comparably on mAP@0.50:0.95 but does not improve over the real-only baseline, and all other metrics are lower. Beyond 20% synthetic, accuracy degrades steadily, and 100% synthetic data results in model collapse. These findings are consistent with the results of Experiment 1 and with the domain gap observed by Vanherle et al. [20]. They suggest that synthetic CCTV data generated by Gemini 2.5 Flash does not provide additional value when sufficient real training data is available.

5 Discussion

This section revisits each research question in turn, using the findings from the four experiments to provide a clear answer. It then discusses the practical implications of these findings for real-world CCTV deployment, followed by the limitations of the study.

5.1 RQ1: Can synthetic data match or supplement real CCTV data?

Experiments 1 and 4 together provide a consistent answer: the synthetic CCTV data used in this study is not an effective substitute for real footage, and it also does not improve performance when added to real data. In Experiment 1, the model trained entirely on synthetic images performed substantially worse than the model trained on real images (mAP@0.50 of 0.719 compared to 0.931), missing almost three times as many people and producing many more false positives. Experiment 4 then tested whether mixing the two sources could combine the visual variety of synthetic images with the realism of real footage. Across eleven different ratios, no mixture outperformed training on real data alone.

The most likely explanation is the domain gap between the two datasets. Real CCTV footage has specific visual characteristics that are difficult to reproduce with a generative model: compression artefacts, camera noise, unusual lighting, and the distinctive elevated viewing angle typical of security cameras. These characteristics shape how the model learns to detect people, and when synthetic images do not capture them accurately the model picks up patterns that do not transfer back to real scenes. This matches the observations of Vanherle et al. [20], who

found that even small visual differences between synthetic and real images can limit how well a model generalises.

It is worth noting that other studies have found synthetic data to be highly effective for person detection in different settings. For example, Fabbri et al. [23] generated a large pedestrian dataset using a video game engine, which they used as the main training source for their detection models. They achieved competitive results on standard pedestrian benchmarks, demonstrating that synthetic data can work well for this task under the right conditions. However, their dataset was far larger and more diverse than the one used in this study, and the camera viewpoints were mostly captured at street level, rather than the elevated overhead angles typical of CCTV cameras. This suggests that the value of synthetic data depends strongly on how closely it matches the intended deployment environment, including factors such as camera angle, scene complexity, and dataset size. For the narrow domain of CCTV person detection, current generative models do not yet produce images that are similar enough to real footage to replace or meaningfully supplement it.

5.2 RQ2: Which edge-optimised architecture provides the best accuracy?

Experiment 2 compared three CNN-based YOLO models (YOLO11n, YOLOv10n, and YOLO26n) with a transformer-based detector (RF-DETR-N). All three YOLO models achieved closely matched performance, each scoring above 0.92 mAP@0.50 on the real CCTV test set. YOLO26n achieved the highest localisation quality (mAP@0.50:0.95 of 0.745), YOLOv10n achieved the highest precision, and YOLO11n achieved the highest recall. The differences between the three were small, showing that all three are well suited to this task.

RF-DETR-N, in contrast, underperformed across all metrics. Its recall of 0.770 was notably lower than the YOLO models, meaning it missed more people. The most likely reason is the size of the dataset. RF-DETR-N has roughly twelve times more parameters than the YOLO nano models, and transformer architectures generally need much larger training datasets to reach their full potential [29]. Chen et al. [41] showed that as the training set shrinks, the accuracy of vision transformers declines much faster than that of CNNs, because they lack the convolutional structure that helps CNNs generalise from limited examples. The original DETR detector was trained on over 118,000 images [42], whereas this experiment used only 1,795. With so little data, a transformer with 30.1 million parameters struggles to learn patterns that transfer to new scenes.

Another important factor is the simplicity of the task. CCTV person detection involves only one type of object (a person), recorded from fixed camera viewpoints with consistent backgrounds. Transformers are designed to consider the entire image at once, which is useful in complex scenes where the model needs to understand the wider context. For example, in a busy street scene, a transformer can use the presence of a road, traffic lights, and other vehicles to help recognise that a small object in the distance is likely to be a car. In a CCTV setting where the only goal is to detect people in a fixed scene, this kind of wider context provides little benefit. CNN-based models, which detect local visual patterns such as edges and shapes, are well matched to the

task and work efficiently even with limited training data.

5.3 RQ3: What is the impact of INT8 quantization on accuracy and inference speed?

Experiment 3 showed that quantization delivers substantial benefits for edge deployment with only minor accuracy loss. Across the three YOLO models, converting from 32-bit to 8-bit representation reduced model file sizes by 47–52%, improved inference speed by 1.56–1.78 times on the Raspberry Pi 5, and reduced mAP@0.50:0.95 by only 1–3%. In one case (YOLOv10n), accuracy after quantization was slightly higher than before. Recent work by Bouguerra et al. [44] has shown that quantization is not always harmful to accuracy and can in some cases improve generalisation by pushing the model to rely on more robust features. While that study examined a different type of model, it suggests that small accuracy gains after quantization, like the one observed here, are not unprecedented.

YOLO26n benefited the most from this process. It achieved the largest size reduction (52%), the fastest inference speed on the Raspberry Pi 5 (5.32 FPS), and retained the highest mAP@0.50:0.95 of all quantized models (0.712). This is consistent with how the architecture was designed, with specific changes aimed at producing clean exports and efficient deployment [39]. The result is a model that is small enough to fit comfortably on low-cost hardware, fast enough to process several frames per second in real time, and accurate enough to reliably detect people in CCTV footage.

5.4 Practical Implications

Taken together, these findings point to a clear recommendation for affordable CCTV analytics. A YOLO26n model, trained on real CCTV data and quantized to INT8, can run directly on a Raspberry Pi 5 at 5.32 frames per second with strong detection accuracy. This has several practical benefits. First, it allows small organisations and home users to perform intelligent video analysis without the cost of dedicated AI hardware or cloud services. Second, because all processing happens on the device, video footage never leaves the premises, which protects privacy and avoids the data protection concerns raised by cloud-based systems. Third, the entire setup can be deployed for a small fraction of the cost of commercial AI-enabled Network Video Recorders, while still providing accuracy that comfortably exceeds simple motion detection and many embedded NVR models.

For practical surveillance tasks such as detecting when a person enters a monitored area, 5 FPS is more than sufficient. Typical human motion is slow enough that the model has multiple opportunities to detect each person, and the low latency of local processing means events can be flagged immediately without waiting for a network round trip.

5.5 Limitations

Several limitations should be acknowledged. First, the real CCTV dataset used in this study was collected from a limited number of camera viewpoints, and although careful steps were

taken to prevent data leakage between splits, the variety of environments is smaller than would be ideal. Results might change if the model were tested on camera types, weather conditions, or locations that were not represented in the training data.

Second, the study focused exclusively on single-class person detection. The conclusions about architecture selection and synthetic data may not generalise to tasks involving multiple object categories such as vehicles, animals, or packages.

Third, the synthetic dataset was generated using a single generative model (Gemini 2.5 Flash). A different model, or a different generation pipeline, might produce synthetic images with a smaller domain gap and therefore different results in Experiments 1 and 4.

Finally, all models were quantized using post-training quantization, which applies compression after the model has already been trained. Quantization-aware training, which simulates quantization during training itself, could reduce the small accuracy drop observed in Experiment 3 and produce even better edge-deployed models.

6 Conclusion

This study set out to evaluate how training data sources, model architecture, and INT8 quantization affect person detection on CCTV footage when the final model is deployed on a Raspberry Pi 5. Three research questions were investigated through four experiments, all using the same real CCTV dataset and a consistent evaluation pipeline.

The first finding is that real CCTV data is essential. Models trained on synthetic CCTV images alone performed substantially worse on real footage, and mixing synthetic data with real data at any proportion failed to improve performance over training on real images alone. The domain gap between current generative-model-produced images and real CCTV footage remains too large for synthetic data to serve as a useful substitute or supplement in this setting.

The second finding is that lightweight CNN-based models are better suited to small CCTV datasets than transformer-based detectors. The three YOLO nano models tested all achieved strong, closely matched performance, while the larger transformer-based RF-DETR-N underperformed across all metrics due to its higher parameter count and the limited size of the training data.

The third finding is that INT8 quantization is highly effective for edge deployment. It reduced model file sizes by approximately half and improved inference speed by up to 1.78 times, with only a small accuracy cost. Among the three quantized models, YOLO26n achieved the best balance of accuracy, speed, and size on the Raspberry Pi 5.

Combining these findings, the recommended configuration for affordable, privacy-friendly CCTV person detection on edge hardware is YOLO26n, trained on real CCTV data and quantized to INT8. This configuration runs at 5.32 frames per second on a Raspberry Pi 5, requires only 4.6 megabytes of storage, and retains strong detection accuracy. It offers a practical, low-cost alternative to cloud-based or expensive on-camera AI systems, making intelligent video analytics

accessible to small organisations and individual users without sacrificing performance or privacy.

7 Future Work

Several directions could extend the findings of this study.

Alternative generative models. The synthetic data in this study was produced using a single generative model (Gemini 2.5 Flash). Different models such as Stable Diffusion or DALL-E may produce synthetic CCTV images with a smaller domain gap, which could improve the effectiveness of synthetic data as a training supplement.

Multi-class detection. This study focused exclusively on single-class person detection. Extending the evaluation to include additional categories such as vehicles or animals would test whether the findings on architecture selection, quantization, and data mixing generalise to more complex detection tasks.

Hardware acceleration. All Raspberry Pi 5 benchmarks in this study used ONNX Runtime on the CPU without dedicated acceleration. The Raspberry Pi AI HAT+ provides a neural network accelerator that could significantly increase inference speed. Evaluating the same models with hardware acceleration would give a more complete picture of what is achievable on current edge hardware.

Larger and more diverse datasets. The real CCTV dataset used in this study was collected from a limited number of camera locations. Expanding the dataset to include more environments, weather conditions, and camera types would strengthen the conclusions and improve the generalisability of the trained models.

Quantization-aware training. This study used post-training quantization, which applies quantization after the model has already been trained. Quantization-aware training (QAT) simulates quantization during the training process itself, allowing the model to learn parameters that are more robust to the precision loss introduced by INT8 conversion. QAT could reduce the accuracy drop observed in Experiment 3 and further improve the quality of edge-deployed models.

References

[1] C. J. N. Cheltha and C. Sharma, "Motion Detection of Human on Video: State of the Art," in *Artificial Intelligence on Medical Data* (M. Gupta, S. Ghatak, A. Gupta, and A. L. Mukherjee, eds.), (Singapore), pp. 471–481, Springer Nature, 2023.

[2] N. A. M. Aris and S. S. Jamaian, "Background subtraction challenges in motion detection using Gaussian mixture model: A survey," *IAES International Journal of Artificial Intelligence (IJ-AI)*, vol. 12, no. 3, pp. 1007–1018, 2023.

[3] "Axis object analytics — axis communications." Accessed: 2025-10-25.

[4] M. Hu, Z. Luo, A. Pasdar, Y. C. Lee, Y. Zhou, and D. Wu, "Edge-Based Video Analytics: A Survey," *arXiv preprint arXiv:2303.14329*, Mar. 2023. arXiv:2303.14329 [cs].

[5] S. Yi, Z. Hao, Q. Zhang, Q. Zhang, W. Shi, and Q. Li, "LAVEA: latency-aware video analytics on edge computing platform," in *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*, (San Jose California), pp. 1–13, ACM, Oct. 2017.

[6] A. C. Cob-Parro, C. Losada-Gutiérrez, M. Marrón-Romera, A. Gardel-Vicente, and I. Bravo-Muñoz, "Smart video surveillance system based on edge computing," *Sensors*, vol. 21, no. 9, p. 2958, 2021.

[7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, pp. 30–39, Jan. 2017.

[8] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 637–646, Oct. 2016.

[9] A. Jaleel, S. K. Khurshid, R. Mustafa, K. Mehmood Aamir, M. Tahir, and A. Ziar, "Towards Proactive Surveillance through CCTV Cameras under Edge-Computing and Deep Learning," *Mathematical Problems in Engineering*, vol. 2022, pp. 1–11, July 2022.

[10] J. Song and J. Nang, "Pedestrian Abnormal Behavior Detection System Using Edge–Server Architecture for Large–Scale CCTV Environments," *Applied Sciences*, vol. 14, p. 4615, May 2024.

[11] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, "Object Detection with Deep Learning: A Review," Apr. 2019. arXiv:1807.05511 [cs].

[12] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, USA), pp. 779–788, IEEE, June 2016.

[14] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, and K. Murphy, "Speed/accuracy trade-offs for modern convolutional object detectors," Apr. 2017. arXiv:1611.10012 [cs].

[15] D. K. Alqahtani, A. Cheema, and A. N. Toosi, "Benchmarking deep learning models for object detection on edge computing devices," *arXiv preprint arXiv:2409.16808*, 2024.

[16] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," Dec. 2017. arXiv:1712.05877 [cs].

[17] S. Boddu and A. Mukherjee, "Lightweight Object Detection Using Quantized YOLOv4-Tiny for Emergency Response in Aerial Imagery," *arXiv preprint arXiv:2506.09299*, June 2025. arXiv:2506.09299 [cs].

[18] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical Text-Conditional Image Generation with CLIP Latents," Apr. 2022. arXiv:2204.06125 [cs].

[19] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. Denton, S. K. S. Ghasemipour, B. K. Ayan, S. S. Mahdavi, R. G. Lopes, T. Salimans, J. Ho, D. J. Fleet, and M. Norouzi, "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding," May 2022. arXiv:2205.11487 [cs].

[20] B. Vanherle, S. Moonen, F. V. Reeth, and N. Michiels, "Analysis of training object detection models with synthetic data," in *Proceedings of the British Machine Vision Conference (BMVC)*, BMVA Press, 2022. Also available as arXiv:2211.16066.

[21] S. Hinterstoisser, V. Lepetit, P. Wohlhart, and K. Konolige, "On Pre-trained Image Features and Synthetic Images for Deep Learning," in *Computer Vision – ECCV 2018 Workshops* (L. Leal-Taixé and S. Roth, eds.), vol. 11129, pp. 682–697, Cham: Springer International Publishing, 2019. Series Title: Lecture Notes in Computer Science.

[22] F. E. Nowruzi, P. Kapoor, D. Kolhatkar, F. A. Hassanat, R. Laganiere, and J. Rebut, "How much real data do we actually need: Analyzing object detection performance using synthetic and real data," July 2019. arXiv:1907.07061 [cs].

[23] M. Fabbri, G. Braso, G. Maugeri, O. Cetintas, R. Gasparini, A. Osep, S. Calderara, L. Leal-Taixe, and R. Cucchiara, "MOTSynth: How Can Synthetic Data Help Pedestrian Detection and Tracking?," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, (Montreal, QC, Canada), pp. 10829–10839, IEEE, Oct. 2021.

[24] A. Voulodimos, N. Doulamis, A. Doulamis, and E. Protopapadakis, "Deep learning for computer vision: A brief review," *Computational Intelligence and Neuroscience*, vol. 2018, no. 1, p. 7068349, 2018.

[25] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *Computer Vision – ECCV 2014* (D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, eds.), (Cham), pp. 740–755, Springer International Publishing, 2014.

[26] R. Padilla, S. L. Netto, and E. A. B. da Silva, "A survey on performance metrics for object-detection algorithms," in *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, pp. 237–242, 2020.

[27] M. Hossin and M. N. Sulaiman, "A review on evaluation metrics for data classification evaluations," *International Journal of Data Mining & Knowledge Management Process*, vol. 5, no. 2, pp. 1–11, 2015.

[28] J. Gupta, S. Pathak, and G. Kumar, "Deep Learning (CNN) and Transfer Learning: A Review," *Journal of Physics: Conference Series*, vol. 2273, p. 012029, May 2022. Publisher: IOP Publishing.

[29] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[30] G. Jocher, A. Chaurasia, and J. Wang, "Ultralytics yolo," 2023.

[31] Vast.ai, "Vast.ai: Gpu rental and cloud computing platform," 2025.

[32] V. Uhunmwangho, "Analysis of pixel-level algorithms for video surveillance applications," master's thesis, University of Oulu, Oulu, Finland, 2021. Available at `https://www.researchgate.net/publication/analysis_of_pixel-level_algorithms_for_video_surveillance_applications`.

[33] G. Comanici, E. Bieber, M. Schaekermann, and I. P. et al, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025.

[34] CVAT.ai Corporation, "Computer vision annotation tool (cvat)," 2023.

[35] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 815–823, IEEE, 2015.

[36] F. Pizzi, M. Berman, F. Radenović, Y. Wei, R. Timofte, and O. Chum, "A self-supervised descriptor for image copy detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13572–13582, June 2022.

[37] J. Gómez and P.-P. Vázquez, "An empirical evaluation of document embeddings and similarity metrics for scientific articles," *Applied Sciences*, vol. 12, no. 11, 2022.

[38] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "Yolov10: Real-time end-to-end object detection," *Advances in Neural Information Processing Systems*, 2024.

[39] G. Jocher and J. Qiu, "Yolo26: Key architectural enhancements and performance benchmarking for real-time object detection." `https://arxiv.org/abs/2509.25164`, 2025.

[40] P. Robinson and N. Layton, "Rf-detr: Neural architecture search for real-time detection transformers." `https://arxiv.org/abs/2511.09554`, 2025.

[41] X. Chen, C.-J. Hsieh, and B. Gong, "When vision transformers outperform resnets without pre-training or strong data augmentations," in *International Conference on Learning Representations (ICLR)*, 2022.

[42] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European Conference on Computer Vision (ECCV)*, 2020.

[43] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "Detrs beat yolos on real-time object detection," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[44] A. Bouguerra, D. Montoya, A. Gomez-Villa, C. Mraidha, and F. Arnez, "Less precise can be more reliable: A systematic evaluation of quantization's impact on clip beyond accuracy," *arXiv preprint arXiv:2509.21173*, 2025.